

Production Deploy Checklist

A step-by-step runbook for shipping the AMW Performance Evaluation System to production. Demo deploys are simpler — see [Deployment \(Demo\)](#); this document is for the real production rollout.

Conventions: `[ ]` = do it, `→` = command,  = stop-and-verify gate.

0. Pre-flight (before touching prod)

- ☐ All target changes are merged to `main` and CI is green.
- ☐ Local release gate passes:
 - `→ npm run verify` (tsc, unit, e2e, build, audit) — or individually:
 - `→ backend: npm test` (unit) and `npm run test:integration` (needs a test DB)
 - `→ frontend: npm test` (Vitest) and `npm run test:e2e` (Playwright)
- ☐ `npm audit` is clean for production deps (backend + frontend).
- ☐ You have a current **database backup** (see §6 rollback) and know how to restore it.
- ☐ Maintenance window agreed; stakeholders notified.

1. Environment configuration

Copy and fill the production env (never commit real secrets):

- `→ cp .env.prod.example .env.prod`

 The API **refuses to boot in production** unless these are set correctly:

Variable	Requirement
<code>NODE_ENV</code>	<code>production</code>
<code>JWT_SECRET</code>	random, ≥ 32 chars , not a placeholder — <code>→ openssl rand -base64 48</code>
<code>DATABASE_URL</code>	valid Postgres connection URL (prod DB)
<code>CLIENT_URL</code> / <code>CORS_ORIGINS</code>	exact prod origin(s); no * wildcard
<code>METRICS_TOKEN</code>	required in prod (guards <code>/metrics</code>)
<code>POSTGRES_USER/PASSWORD/DB/HOST/PORT</code>	prod database credentials
<code>COMPANY_EMAIL_DOMAIN</code>	e.g. <code>amw-ems.com</code>

- ☐ `.env.prod` filled; secrets stored in the secret manager (not in git).
- ☐ `ALLOW_PROD_SEED` is **unset/false** (the demo seed must not run against prod).

2. Database migration

The deploy ships new schema (scoring weights, calibration grade, session revocation `passwordChangedAt`, self-service `phone` / `bio`, soft-delete `deletedAt`). Migrations are forward-only and idempotent.

-  Confirm `DATABASE_URL` points at **production** (not localhost).
- → `cd backend && npx prisma migrate deploy`
- ☐ Output shows "No pending migrations" or lists the newly applied ones with no error.

3. First-time data / credentials

Skip the parts that don't apply to a re-deploy.

Fresh environment (no users yet):

- → import staff via the Data Management page or `POST /api/users/import`

Migrating staff off the old shared default password (one-time, ~709 users):

- → `cd backend && node scripts/reissue-temp-passwords.cjs --dry-run` (count only)
-  Review the count, then run for real:
- → `node scripts/reissue-temp-passwords.cjs` (writes `temp-credentials-<date>.csv`)
- ☐ Distribute credentials over a **secure channel**, then **delete the CSV**.
- ☐ All affected users keep `mustChangePassword = true` (forced change on first login).

Steps 2 + 3 are bundled in `backend/scripts/deploy-v1.4.3.sh` (guarded against running on a localhost DB). Run it on the prod host:
`./scripts/deploy-v1.4.3.sh`

4. Build & release

Docker (recommended):

- → `docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build`
 - Brings up Postgres, a one-shot migration step, the API, and the nginx-served frontend.
- ☐ (Optional) observability stack: `docker compose -f docker-compose.monitor.yml up -d`

Or CI images: pull `ghcr.io/<owner>/amw-{backend,frontend}` tags published by CI on `main`.

- ☐ Frontend version badge / `APP_VERSION` matches the intended release.

5. Smoke test (post-deploy verification)

 Do not announce go-live until these pass against the **production URL**:

- ☐ → `curl -s <host>/health` → 200 with correct `version`.
- ☐ → `curl -s <host>/api/ready` → 200, database `ok`.
- ☐ → `curl -s -H "Authorization: Bearer $METRICS_TOKEN" <host>/metrics` → 200; without token → 401.
- ☐ Login works for a known admin; landing page loads.
- ☐ One evaluation opens; a report renders; an export downloads.
- ☐ Calibration: an admin can set a final grade (saves).
- ☐ Security headers present: `Cache-Control: no-store` on `/api/*`, no `X-Powered-By`.
- ☐ Browser console is clean on the main pages (no errors).

6. Rollback plan

If a smoke test fails and a fix isn't immediate:

- ☐ **App:** redeploy the previous image tag (or `git revert` + redeploy).
- ☐ **Database:** migrations are forward-only — restore from the pre-deploy backup if a migration caused data issues:
 - → `pg_restore / psql` the backup taken in §0 into the prod DB, then redeploy the prior app version.
- ☐ Note: deletes are now **soft** (`deletedAt`), so accidental deletions in the app are recoverable without a DB restore.

7. Post-deploy

- ☐ Monitor logs/metrics for the first 30–60 min (error rate, latency, 5xx).
- ☐ Confirm audit events are recording (login/export/mutation) with request IDs.
- ☐ Tag the release in git and update the [Changelog](#) if not already.
- ☐ Securely archive (then delete local copies of) any credential CSVs.
- ☐ Close the maintenance window; notify stakeholders.

Quick reference

Item	Where
One-shot prod deploy (migrate + reissue)	<code>backend/scripts/deploy-v1.4.3.sh</code>
Temp-password reissue	<code>backend/scripts/reissue-temp-passwords.cjs</code>
Prod compose	<code>docker-compose.prod.yml</code> + <code>.env.prod</code>
Health / readiness	<code>/health</code> , <code>/api/ready</code>
Metrics (token-guarded)	<code>/metrics</code>
API docs	<code>/api/docs/</code> , <code>/api/openapi.json</code>
Test catalog	system-test-cases.md
Operations runbook	operations-runbook.md